



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Computer Architecture and Assembly Lab (14:332:333) Spring 2026

Lab 4 *RISC-V Registers & Procedures*

Goals

In Lab 4, we will discuss the following topics:

- Review the Venus simulator, including the environmental call and assembler directives.
- RISC-V calling conventions, e.g., register usage.
- Introduction to the function/procedure in RISC-V.

Instructions

- All the programs must be compiled by [Venus RISC-V Simulator](#). Please note that the simulator only supports 32-bit instruction, and we do not consider overflow.
- Once you complete the assignment, please upload your source code and PDF lab report on Canvas.
- **Comment your code clearly. You can use the mark “#” to add comments.**
- Please copy and paste your code into the lab report.

Introduction

Venus Simulator Review

Environmental call (ecall)

Just as 'printf' is used in the C language to display content on the console, 'ecall' serves a similar purpose. Here, the register 'a0' (also known as 'x10') holds the function code, while 'a1' (or 'x11') contains the actual content we aim to display.

Assembler directives

Besides instructions in assembler format, an assembler also accepts so-called assembler directives.

- **.text** – Where the code starts. Each assembler instruction can start with a label such as **main:**.
- **.data** – data segment, where you can initialize data. Also, data can be addressed by using a label.



RISC-V Calling Conventions in Register Usages

In RISC-V programming, the calling convention is a standard that defines how functions/procedures receive parameters and return values. It also determines which registers are used for what purpose and who (caller or callee) is responsible for saving and restoring the contents of registers.

ra (x1) : return address. No need to save & restore

a0–a7 (x10–x17) : registers for procedures

a0, a1 (x10, x11) : procedure arguments/return values

a2–a7 (x12–x17) : procedure arguments. No need to save & restore

t0–t6 (x5–x7, x28–x31) : temporary registers. Not preserved by the callee. No need to save & restore

s0–s11 (x8, x9, x18–x27) : saved registers. Must be preserved by the callee. Need to save & restore

Introduction to RISC-V Functions/Procedures

The function/procedure is a stored subroutine that performs a specific task based on the provided parameters.

Procedure call: jump-and-link, e.g., **jal ra, ProcedureLabel**

- Puts the address of following instruction to **ra**
- Jumps to target address **ProcedureLabel**

Procedure return: jump-and-link register, e.g., **jalr x0, 0(ra)**

- Like **jal**, but jumps to (0 + address in **ra**)
- Use **ra** as the destination register (rd) (**x0** cannot be changed).

Steps required:

1. Place parameters in registers (**a0** to **a7**)
2. Transfer control to procedure
3. Acquire storage resources needed for procedure
4. Perform procedure's operations
5. Place results in registers for the caller (**a0, a1**)
6. Return control to the point of origin (address in **ra**)



Exercise 1 RISC-V Calling Conventions [40 pts]

Please read the C program below and answer the following questions.

```
//ex1.c
#include "stdio.h"

int source[] = {22, 44, 88, 166, 322, 644, 0};

int main() {
    int k;
    int rem = 2000;

    for (k = 0; source[k] != 0; k++) {
        rem = rem - source[k] / 2;
    }

    printf("%d", rem);

    return 0; //no need to translate this line of code
}
```

1.1 [5 pts] In RISC-V, which registers would you use to send data or inputs into procedures when programming?

To pass arguments or inputs in procedures, I would use a0-a7 (also called x10-x17) registers. Using a0 and a1 and procedure arguments and return values while a2-a7 as just procedure arguments.

1.2 [5 pts] In RISC-V, which register(s) should hold the results that are produced by procedures?

The registers liable for holding the result are a0 and a1 (x10 and x11).

1.3 [5 pts] If you are calling a procedure using the 'jal' instruction and you expect to return from that procedure, we need to use the register ra (x1). What kind of value does the 'jal' instruction store in ra (x1)?



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

The jal instruction stores the address of the instruction immediately following the command and stores `l` into the `ra` (which is the `x1`) register.

1.4 [5 pts] If you're modifying the value of '`x8 (s0)`' within a procedure and intend to use this updated value every time the procedure is called. Is it necessary to save the values stored in `x8`? Please write down your explanation.

Yes it is really important that we save the value of `x8(s0)` as in RISC-V, if a procedure modifies an `s` register, it must first back up the original value before returning the data to the client.

1.5 [20 pts] Please convert the C program into a RISC-V program by following calling conventions. The RISC-V program should work the same way as the C program at the function/procedure level, which means the loop from the original program should be maintained. You might need to define a '`main`' procedure and a '`loop`' procedure to answer this question.

In your report, please include the following:

- (1) A screenshot of your program's output.
- (2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**



1357

```
.data
source: .word 22, 44, 88, 166, 322, 644, 0

.text
main:
    addi x8, x0, 2000    # x8 is the same thing as s0
    la x9, source       # grabbing the array address

    addi x10, x8, 0
    addi x11, x9, 0

    jal x1, loop_proc

    addi x11, x10, 0    #printing the result
    addi x10, x0, 1
    ecall

    addi x10, x0, 10
    ecall

loop_proc:
    addi x5, x0, 0

loop_start:
    slli x6, x5, 2
    add x7, x11, x6
    lw x28, 0(x7)

    beq x28, x0, loop_end

    srai x29, x28, 1
    sub x10, x10, x29
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
addi x5, x5, 1  
jal x0, loop_start
```

```
loop_end:  
jalr x0, x1, 0
```

Exercise 2 Writing RISC-V Procedures [60 pts]

2.1 [20 pts] Please write a program in RISC-V that completes the following requirements:

In the `.data` directive:

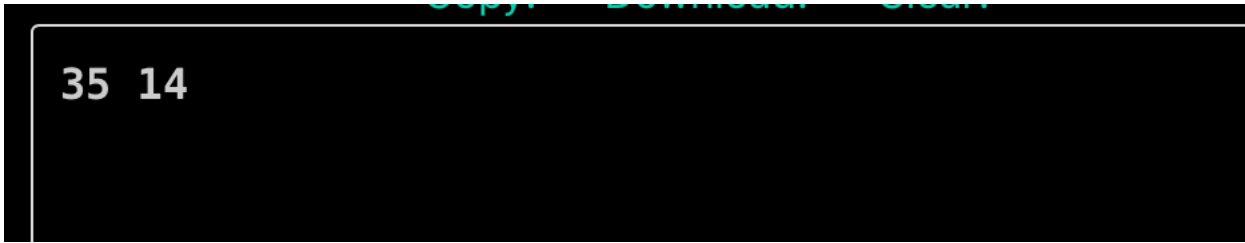
- Define the variables `x` and `y` in the `.word` type.

In the `.text` directive:

- The `main` procedure is designed to retrieve the input values from the `.data` directive and enter the `branch_1` and the `branch_2` respectively. Also, the main procedure prints the computed `x` and `y` in the console using `ecall` after calling the `branch_1` and the `branch_2`. For example, the output could be formatted as: "`x=21 y=14`".
- In the `branch_1` procedure, if `x < 10`, compute `7*x`, else do nothing.
- In the `branch_2` procedure, if `y` is negative, compute `-y`, else do nothing. You can get the opposite value by calculating `0-y`.

In your report, please include the following:

- (1) A screenshot of your program's output. The output should be based on your own input.
- (2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**



```
35 14
```

X = x*7 so x = 35 in the end and y = -14 and computing -y = 14

```
.data  
x_val: .word 5  
y_val: .word -14
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
.text
main:
    la x5, x_val
    lw x8, 0(x5)

    addi x10, x8, 0
    jal x1, m_1
    addi x8, x10, 0    # save x back so it doesn't get lost

    la x5, y_val
    lw x9, 0(x5)

    addi x10, x9, 0
    jal x1, m_2
    addi x9, x10, 0

    addi x11, x8, 0
    addi x10, x0, 4
    ecall

    addi x11, x8, 0    # print the actual x value
    addi x10, x0, 1
    ecall

    addi x11, x9, 0
    addi x10, x0, 4
    ecall

    # print the actual y value
    addi x11, x9, 0
    addi x10, x0, 1
    ecall

    addi x10, x0, 10
    ecall

m_1:
    addi x5, x0, 10
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
bge x10, x5, end_m1  # had to swap ble for bge here since ble is pseudo
addi x6, x0, 7
mul x10, x10, x6
end_m1:
jalr x0, x1, 0

m_2:
bge x10, x0, end_m2
sub x10, x0, x10  # flip the sign if negative
end_m2:
jalr x0, x1, 0
```

2.2 [20 pts] Please write a RISC-V program in Venus simulator that accepts an input integer x and computes the following function:

$$f(x) = x! = x * (x - 1) * (x - 2) * \dots * 2 * 1$$

You can assume x is no bigger than 10.

In the program, please have a main procedure that takes the value of the input, performs the factorial computations, and prints the output in the console.

In your report, please include the following:

- (1) A screenshot of your program's output. **Verify your program with 3 different input values x_1 , x_2 , and x_3 such that $x_1 \neq x_2 \neq x_3$.**
- (2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

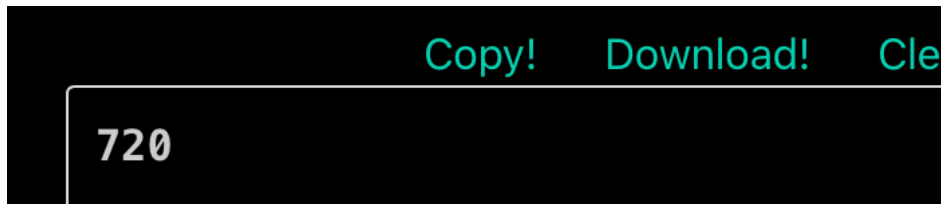
When $x = 5$:



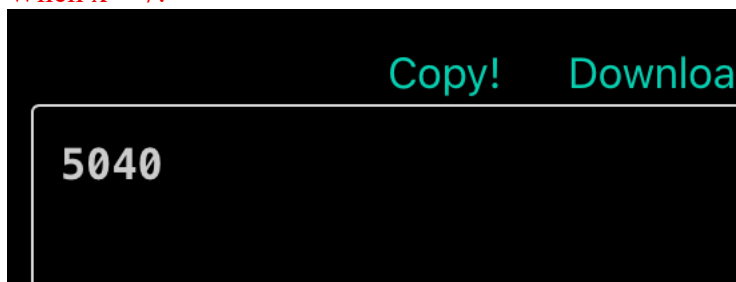
When $x = 6$:



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey



When $x = 7$:



```
.data
x_in: .word 5      # Testing with 5, 6, 7

.text
main:
    la x5, x_in      # Grabbing the memory address of our input variable
    lw x10, 0(x5)     # Loading the actual number into x10 (a0) so we can pass it

    jal x1, snow_depth_calc # Jump to my custom factorial function

    addi x11, x10, 0   # Moving the result from x10 into x11 (a1) to print it
    addi x10, x0, 1
    ecall

    addi x10, x0, 10   # Setting ecall code to 10 (exit program)
    ecall

snow_depth_calc:      # My renamed factorial procedure
    addi x5, x0, 1
    addi x6, x10, 0

depth_loop:
    bge x0, x6, depth_end # If multiplier <= 0, we are done multiplying
    mul x5, x5, x6
    addi x6, x6, -1
```



```
jal x0, depth_loop
```

```
depth_end:
```

```
addi x10, x5, 0    # Putting the final factorial result back into x10 (a0) for return  
jalr x0, x1, 0
```

2.3 [20 pts] In this question, we define an expression:

$$\text{exp}(x) = x^5 + 4x - 7$$

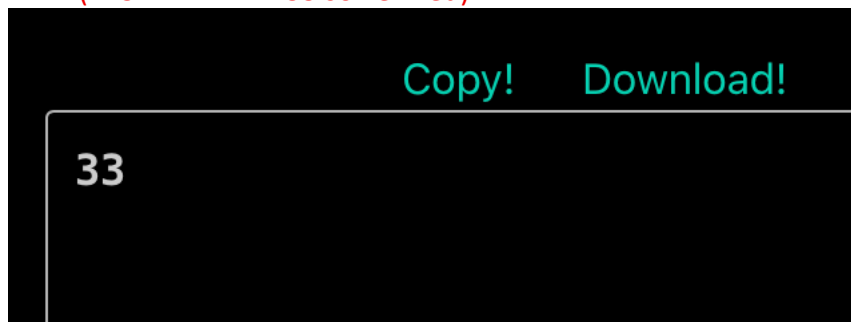
Please write a program in RISC-V that completes the following requirements:

- The **main** procedure is to compute the above expression and print the output in the console. You can assume the input x will always be positive, and hardcode the value of x in the **main** procedure. The **main** procedure must call the following two procedures:
 - Call the **power** procedure to calculate x^n . You can assume that n will always be positive.
 - Call the **times** procedure to calculate $m \cdot x$.

In your report, please include the following:

- (1) A screenshot of your program's output. The output should be based on your own input.
- (2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

$x = 2$ ($2^5 + 2 \cdot 4 - 7 = 33$ so verified)



```
.text
```

```
main:
```

```
addi x8, x0, 2    # x = 2 to test the expression
```

```
addi x10, x8, 0
```

```
addi x11, x0, 5    # Passing the power (5) as the second argument in x11
```



```
    jal x1, lift_power # Jumping to our power calc function (named it this cuz I'm missing my
workout for this lab)
    addi x9, x10, 0    # Saving the x^5 result into x9 so it doesn't get overwritten

    addi x10, x0, 4    # Passing 4 as the first argument in x10
    addi x11, x8, 0
    jal x1, gym_reps

    add x9, x9, x10    # Adding the result of 4*x to our saved x^5 (x9 = x9 + x10)
    addi x9, x9, -7    # Subtracting 7 from the total sum

    addi x11, x9, 0    # Move the final expression result into x11 for printing
    addi x10, x0, 1
    ecall

    addi x10, x0, 10
    ecall
lift_power:          # Function to calculate x^n
    addi x5, x0, 1
lift_loop:
    beq x11, x0, lift_end # If the exponent hits 0, break out of the loop
    mul x5, x5, x10
    addi x11, x11, -1    # Subtract 1 from the exponent
    jal x0, lift_loop    # Loop back around
lift_end:
    addi x10, x5, 0
    jalr x0, x1, 0      # Return to main (strict base format)

gym_reps:           # Function to calculate m * x
    mul x10, x10, x11
    jalr x0, x1, 0      # Return back to main
```